Go

Some Algorithms and
Data structures

Binary tree

```
type Tree struct {
        Left  *Tree
        Value int
        Right *Tree
}

func NewTree(v int) *Tree {

        return &Tree{nil, v, nil}
}

func (t *Tree) InOrder() {

        if t.Left != nil {
          t.Left.InOrder()
        }
        fmt.Println(t.Value)
        if t.Right != nil {
          t.Right.InOrder()
        }
}
```

Inserting in a binary search tree

```
func (t *Tree) Insert(v int) *Tree {

        if v < t.Value {
                if t.Left == nil {
                        t.Left= NewTree(v)
                        return t.Left
                } else {
                        return t.Left.Insert(v)
                }
        } else {
                if t.Right == nil {
                        t.Right= NewTree(v)
                        return t.Right
                } else {
                        return t.Right.Insert(v)
                }
        }
        return t
}
```

```
func main() {

    t:= NewTree(5)

    t.Insert(7)
    t.Insert(9)
    t.Insert(2)

    t.InOrder()
}
```

A generic queue

```
// any is a keyword that designates any type i.e. interface{}
type Queue[T any] struct {
    items []T
}

// adds a value to the end of the queue
func (q *Queue[T]) Enqueue(item T) {

    q.items = append(q.items, item)
}

// removes from the start of the queue
func (q *Queue[T]) Dequeue() T {

    item := q.items[0]
    q.items = q.items[1:]
    return item
}
```

Parallel Quicksort

```
func qsort_pass(arr []int, done chan int) []int{
    if len(arr) < 2 {
        done <- len(arr)
        return arr
    }
    pivot := arr[0]
    i, j := 1, len(arr)-1
    for i != j {
        for arr[i] < pivot && i!=j{
            i++
        }
        for arr[j] >= pivot && i!=j{
            j--
        }
        if arr[i] > arr[j] {
            arr[i], arr[j] = arr[j], arr[i]
        }
    }
    if arr[j] >= pivot {
        j--
    }
    arr[0], arr[j] = arr[j], arr[0]
    done <- 1;

    go qsort_pass(arr[:j], done)
    go qsort_pass(arr[j+1:], done)
    return arr
}
```

```
func qsort(arr []int) []int {
    done := make(chan int)
    defer func() {
        close(done)
    }()

    go qsort_pass(arr[:], done)

    rslt := len(arr)
    for rslt > 0 {
        rslt -= <-done;
    }
    return arr
}
```